

NIP Project — Submission Template

Pega Platform project · This document is your complete submission

Read this before you start:

1. Fill in Section 1 completely. The Pega application name must follow the format NIP-[Project]-[YourName].
2. Create a new Operator in Pega with your full name and log in as that operator before taking any screenshot.
3. When you create your test case, enter YOUR FULL NAME in the Issue Description (Vehicle) or Movie Name (Movie) field.
4. Complete only the section for your chosen project. Delete the other project section.
5. Paste one screenshot per user story (10 total). Follow the screenshot rules in your selected project section.
6. Answer every question in Sections 3 and 4 in your own words.
7. Save the file using this format: ProjectName_FirstName_LastName.docx. Use VehicleService or MovieTicket as the ProjectName. Examples: VehicleService_Ravi_Kumar.docx or MovieTicket_Ravi_Kumar.docx.

Section 1 — Your Details

Full Name	Rishvin Reddy
Email ID (registered on NIP)	rishvinreddy@gmail.com
Phone Number	9848723235
College Name	Woxsen University
Course (e.g. IT, CS)	Computer Science (CS)
State	Telangana, India
Project Chosen (Vehicle Service OR Movie Ticket Booking)	Movie Ticket Booking
Pega Application Name (e.g. NIP-VehicleService-YourName)	NIP-MovieTicket-RishvinReddy
Case Type Name (exact)	Movie Ticket Request
Pega Instance URL	https://2w6cjkfd.pegacea.net/prweb
Operator Name (created in Pega with your full name)	Rishvin Reddy

Section 2A — Vehicle Service Management Application

Complete this section only if you selected this project. Delete the other project section before submitting.

Screenshot rules — IMPORTANT before you start:

1. Name your Pega application as: NIP-VehicleService-YourName OR NIP-MovieTicket-YourName
2. Create a new Operator in Pega with your full name (Security → Operators) and log in as that operator.
3. When you create your test case, enter YOUR FULL NAME in the Issue Description (Vehicle) or Movie Name (Movie) field.
4. The logged-in operator name must be visible in every screenshot wherever the Pega interface displays it.
5. The student's full name must be visible in all running-case screenshots.
6. Configuration screenshots must clearly show the relevant application or rule name.

Screenshots that do not show these will not be accepted.

US-001 — Submit Vehicle Service Request

What is expected:

Case created, customer submits Vehicle ID, Vehicle Model, Issue Description. Validation applied.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-002 — Perform Vehicle Inspection

What is expected:

Inspection stage with Service Advisor step. Inspection Notes and Condition Rating captured.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-003 — Generate Service Estimate

What is expected:

Labor Cost and Parts Cost captured. Total Cost calculated automatically by declare expression.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-004 — Approve Service Estimate

What is expected:

Approval stage assigned to Customer. Approval Status captured. Flow branches on approve/reject.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-005 — Maintain Vehicle Data

What is expected:

Reusable Vehicle data object created with Vehicle ID, Model, Type — linked to case.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-006 — Review Service Estimate

What is expected:

Approval screen shows Labor Cost, Parts Cost and Total Cost clearly to the Customer persona.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-007 — Auto Assign Technician

What is expected:

Service Execution stage with routing to HeavyVehicleQueue or LightVehicleQueue. Assigned Technician and Service Status properties maintained.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-008 — Notify Service Completion

What is expected:

Correspondence rule created and triggered on case resolution. Email includes case and vehicle details.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-009 — Define Service SLA

What is expected:

SLA configured on the case type. Goal: 2 days. Deadline: 3 days. Priority increases on breach.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

US-010 — Route by Vehicle Type

What is expected:

Routing rule or Decision Table routes to HeavyVehicleQueue or LightVehicleQueue based on Vehicle Type property.

Screenshot — paste your screenshot below this line:

What I built for this (1–2 lines in your own words):

Section 2B — Movie Ticket Booking Management Application

Complete this section only if you selected this project. Delete the other project section before submitting.

Screenshot rules — IMPORTANT before you start:

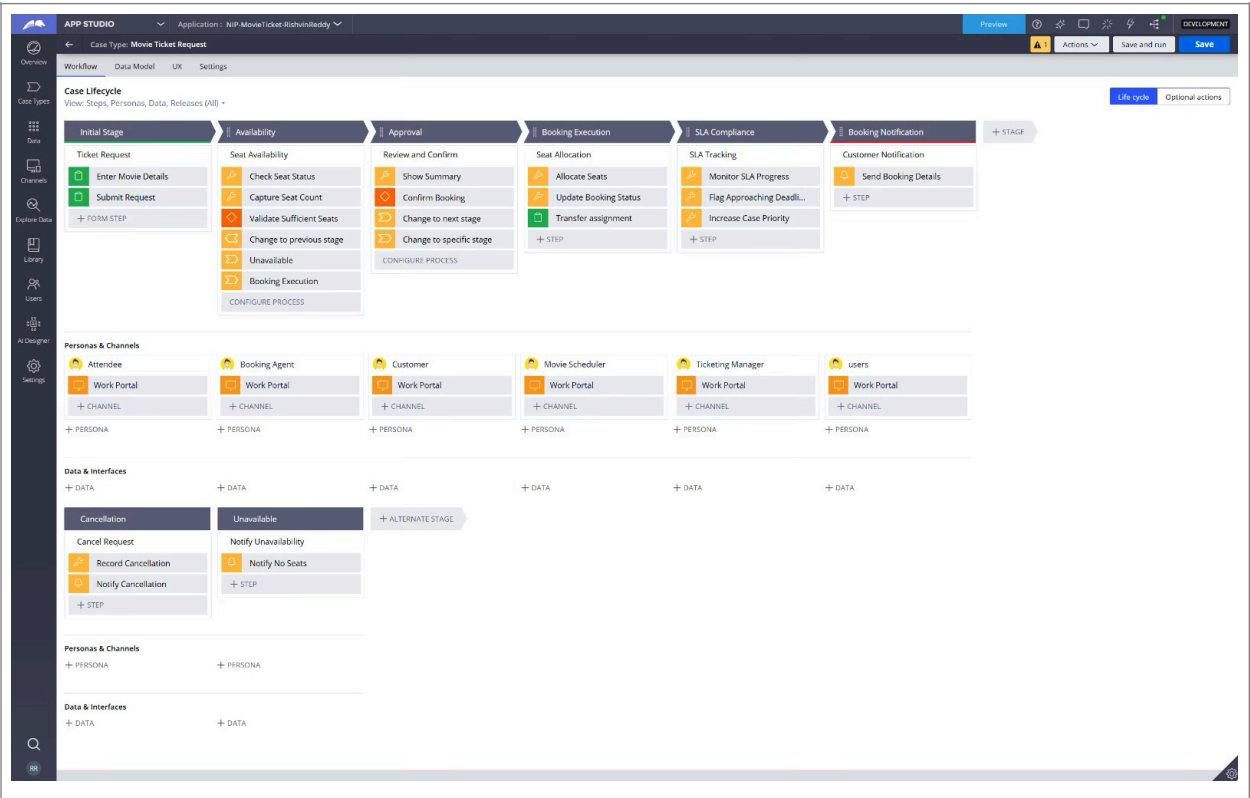
1. Name your Pega application as: NIP-VehicleService-YourName OR NIP-MovieTicket-YourName
 2. Create a new Operator in Pega with your full name (Security → Operators) and log in as that operator.
 3. When you create your test case, enter YOUR FULL NAME in the Issue Description (Vehicle) or Movie Name (Movie) field.
 4. The logged-in operator name must be visible in every screenshot wherever the Pega interface displays it.
 5. The student's full name must be visible in all running-case screenshots.
 6. Configuration screenshots must clearly show the relevant application or rule name.
- Screenshots that do not show these will not be accepted.

US-001 — Submit Movie Ticket Request

What is expected:

Case created, customer submits Movie Name, Show Date, Show Time, Number of Tickets. Validation applied.

Screenshot — paste your screenshot below this line:



What I built for this (1–2 lines in your own words):

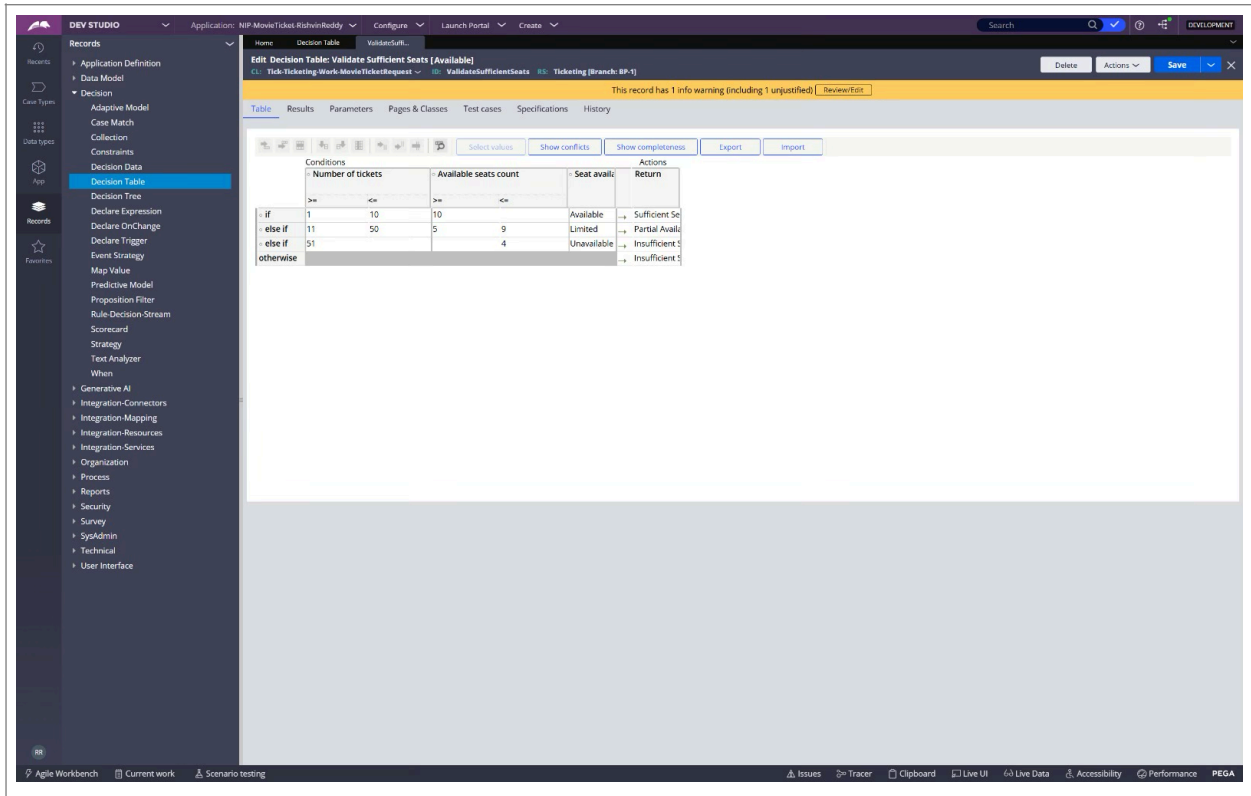
I created the Movie Ticket Request case to capture Movie Name, Show Date, Show Time, and Number of Tickets. I also configured validation for the ticket request details.

US-002 — Check Show Availability

What is expected:

Availability stage. Seat Availability Status and Available Seats Count captured. Booking proceeds only if available.

Screenshot — paste your screenshot below this line:



What I built for this (1–2 lines in your own words):

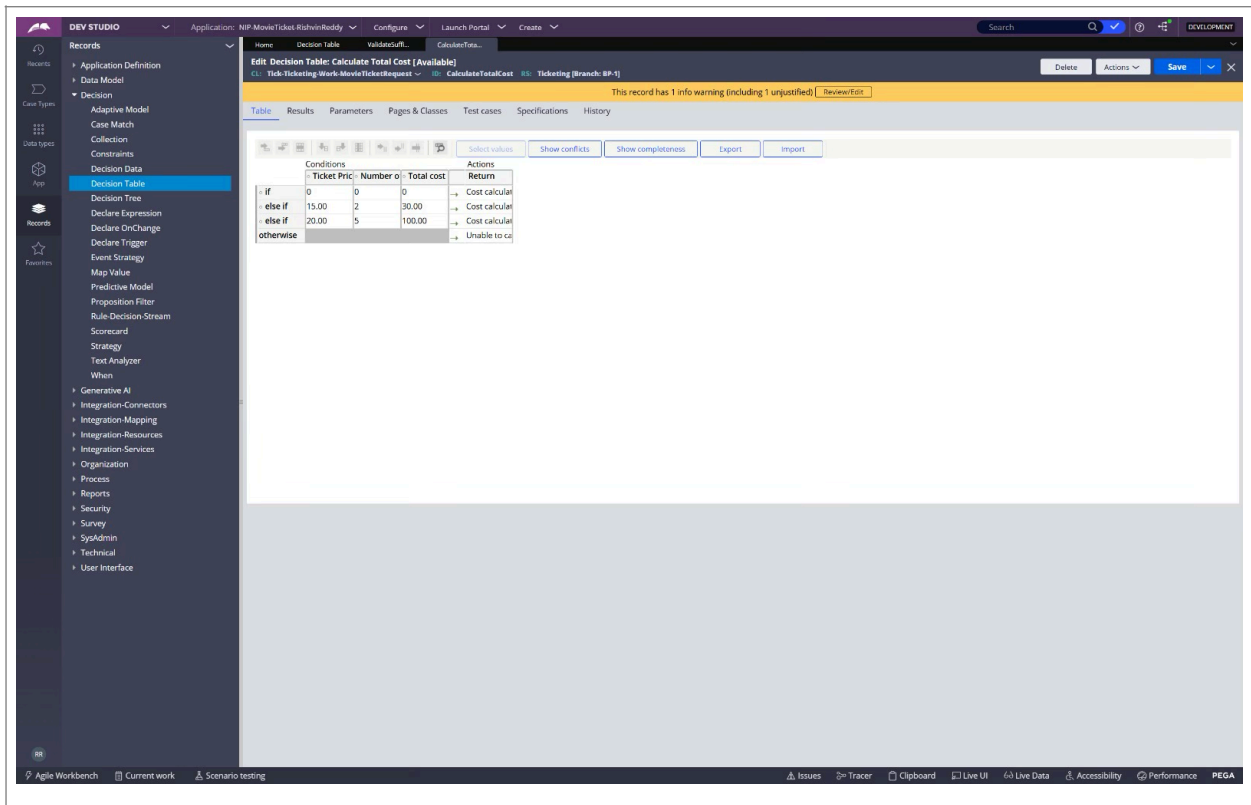
I created an Availability stage that captures Seat Availability Status and Available Seats Count. The case uses this information to determine whether booking can proceed.

US-003 — Calculate Booking Cost

What is expected:

Total Cost calculated automatically from Ticket Price and Number of Tickets using a business rule.

Screenshot — paste your screenshot below this line:



What I built for this (1–2 lines in your own words):

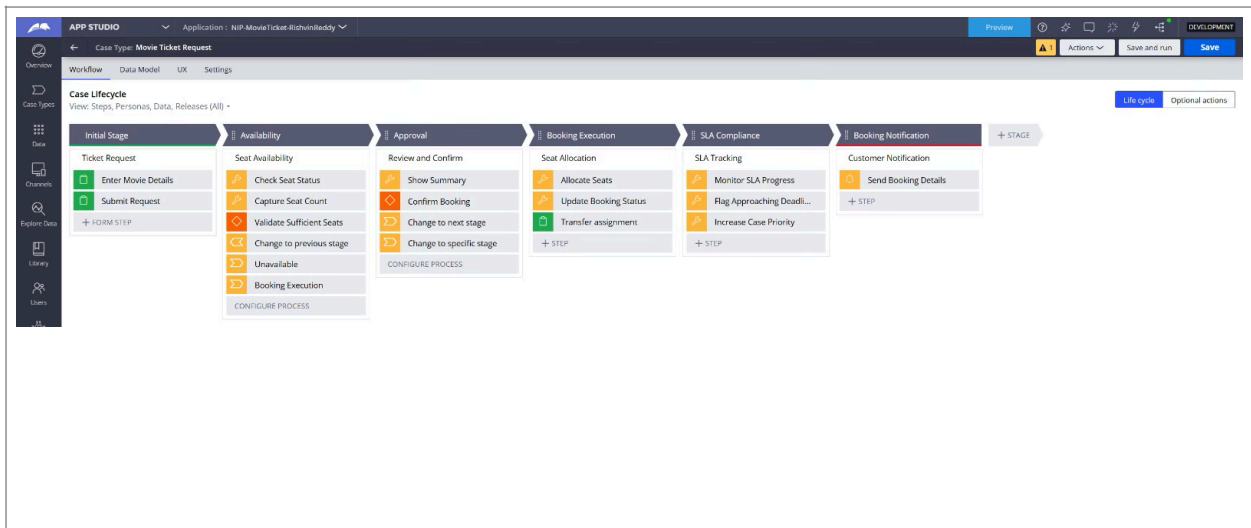
I configured Total Cost to be calculated automatically from Ticket Price and Number of Tickets using a business rule.

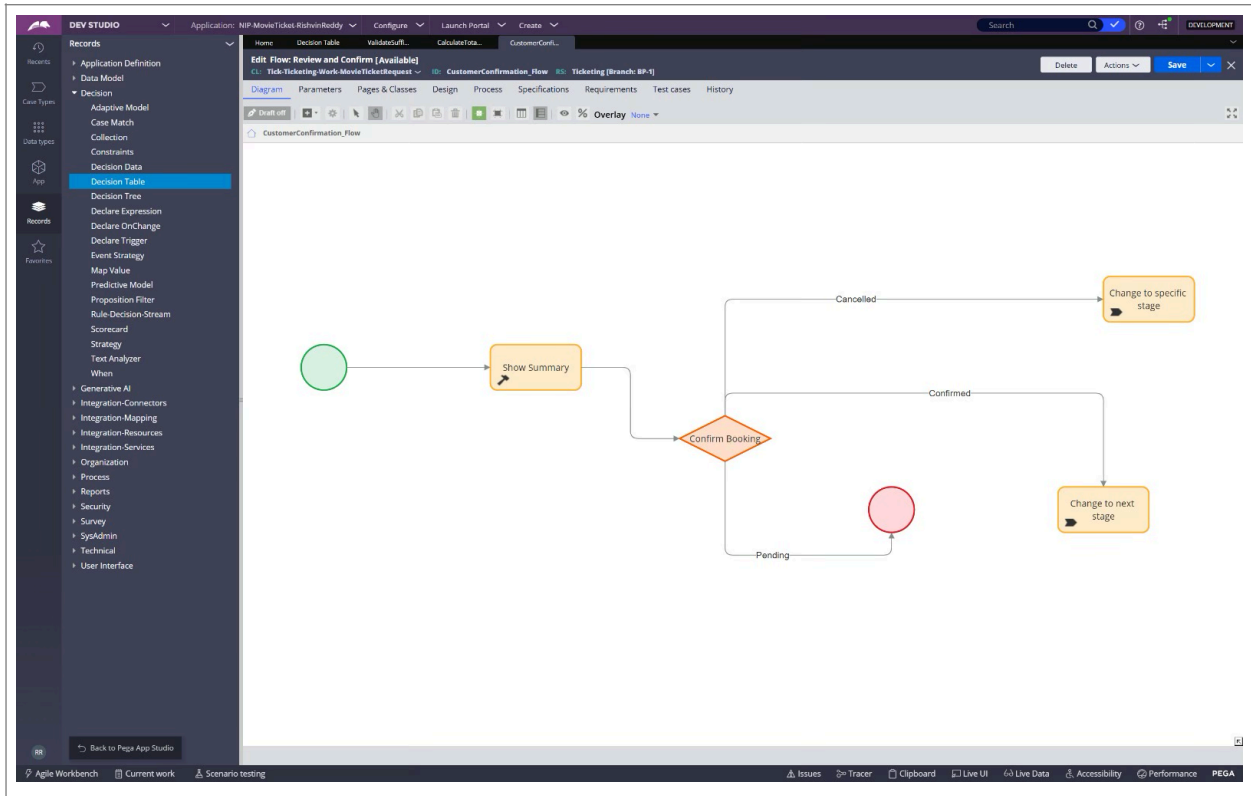
US-004 — Confirm Booking Request

What is expected:

Approval stage assigned to Customer. Booking Status captured. Flow branches on confirm/cancel.

Screenshot — paste your screenshot below this line:





What I built for this (1–2 lines in your own words):

I created the Customer approval step to capture the booking decision and Booking Status. The flow branches based on whether the customer confirms or cancels the booking.

US-005 — Maintain Movie and Show Data

What is expected:

Reusable Movie and Show data objects created with Movie Name, Genre, Show Date, Show Time, Seat Capacity — linked to case.

Screenshot — paste your screenshot below this line:

APP STUDIO

Application : NIP-MovieTicket-RishvinReddy

Preview

DEVELOPMENT

←

Data object: Movie

Systems of Record: Local

Save

Overview

Case Types

Data

Channels

Explore Data

Library

Users

AI Designer

Settings

Data Model

Records

UX

Data Pages

Actions

Settings

Search...

Q

Show system fields

View Data Model

Primary Fields

Add Field

Name	ID	Type	Options	Application Layer
Country by Code	pyCountryCode	Text (single line)		Pega Platform
Genre	Genre	Text (single line)		NIP-MovieTicket-RishvinReddy
Globally unique ID	pyGUID	Text (single line)	Key (autogenerated); Read-only	Pega Platform
Movie Name	MovieName	Text (single line)		NIP-MovieTicket-RishvinReddy

APP STUDIO

Application : NIP-MovieTicket-RishvinReddy

Preview

DEVELOPMENT

←

Data object: Show

Systems of Record: Local

Save

Overview

Case Types

Data

Channels

Explore Data

Library

Users

AI Designer

Settings

Data Model

Records

UX

Data Pages

Actions

Settings

Search...

Q

Show system fields

View Data Model

Primary Fields

Add Field

Name	ID	Type	Options	Application Layer
Available seats count	AvailableSeatsCount	Integer		NIP-MovieTicket-RishvinReddy
Country by Code	pyCountryCode	Text (single line)		Pega Platform
Globally unique ID	pyGUID	Text (single line)	Key (autogenerated); Read-only	Pega Platform
Movie	Movie	Data Reference	Movie	NIP-MovieTicket-RishvinReddy
Premium show queue	PremiumShowQueue	Data Reference	Premium Show Queue	NIP-MovieTicket-RishvinReddy
Seat capacity	SeatCapacity	Integer		NIP-MovieTicket-RishvinReddy
Show date	ShowDate	Date only		NIP-MovieTicket-RishvinReddy
Show label	ShowLabel	Text (single line)		NIP-MovieTicket-RishvinReddy
Show time	ShowTime	Time only		NIP-MovieTicket-RishvinReddy
Show type	ShowType	Picklist	Drop-down list	NIP-MovieTicket-RishvinReddy
Standard show queue	StandardShowQueue	Data Reference	Standard Show Queue	NIP-MovieTicket-RishvinReddy
Theatre location	TheatreLocation	Text (single line)		NIP-MovieTicket-RishvinReddy
Ticket price	TicketPrice	Currency		NIP-MovieTicket-RishvinReddy

APP STUDIO Application: NIP-MovieTicket-RishvinReddy

Case Type: Movie Ticket Request

Search... Show system fields

View Data Model Primary Fields Add Field

Name	ID	Type	Options	Application Layer
Available seats count	AvailableSeatsCount	Integer		NIP-MovieTicket-RishvinReddy
Booking confirmation status	BookingConfirmationStatus	Picklist	Drop-down list	NIP-MovieTicket-RishvinReddy
Case ID	pyID	Text (single line)	Key; Read-only	Pega Platform PRIMARY
Case priority	CasePriority	Text (single line)	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
Country by Code	pyCountryCode	Text (single line)		Pega Platform
Customer	Customer	Data Reference	Customer	NIP-MovieTicket-RishvinReddy PRIMARY
Customer confirmation	CustomerConfirmation	Text (single line)		NIP-MovieTicket-RishvinReddy
Customer Email	CustomerEmail	Email		NIP-MovieTicket-RishvinReddy
Description	pyDescription	Text (paragraph)	Plain text	Pega Platform
Duplicate Case Reference	pyDuplicateCaseReference	Query	Case	Pega Platform
Label	pyLabel	Text (single line)		Pega Platform PRIMARY
Movie	Movie	Data Reference	Movie	NIP-MovieTicket-RishvinReddy PRIMARY
Movie ticket request name	MovieTicketRequestName	Text (single line)		NIP-MovieTicket-RishvinReddy PRIMARY
Notes	pyNote	Text (paragraph)	Plain text	Pega Platform
Notification status	NotificationStatus	Text (single line)	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
Number of Tickets	NumberOfTickets	Integer		NIP-MovieTicket-RishvinReddy
Seat Availability Status	SeatAvailabilityStatus	Text (single line)		NIP-MovieTicket-RishvinReddy
Show	Show	Data Reference	Show	NIP-MovieTicket-RishvinReddy PRIMARY
Show type	ShowType	Picklist	Drop-down list	NIP-MovieTicket-RishvinReddy
SLA deadline date	SLADeclineDate	Text (single line)	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
SLA flag status	SLAFlagStatus	Text (single line)	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
SLA goal date	SLAGoalDate	Text (single line)	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
Ticket ID	TicketID	Text (single line)	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
Ticket Price	TicketPrice	Currency		NIP-MovieTicket-RishvinReddy PRIMARY
Ticket Quantity	TicketQuantity	Text (single line)	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
Total cost	TotalCost	Currency	Calculated; Read-only	NIP-MovieTicket-RishvinReddy
Work status	pyStatusWork	Text (single line)		Pega Platform PRIMARY

What I built for this (1–2 lines in your own words):

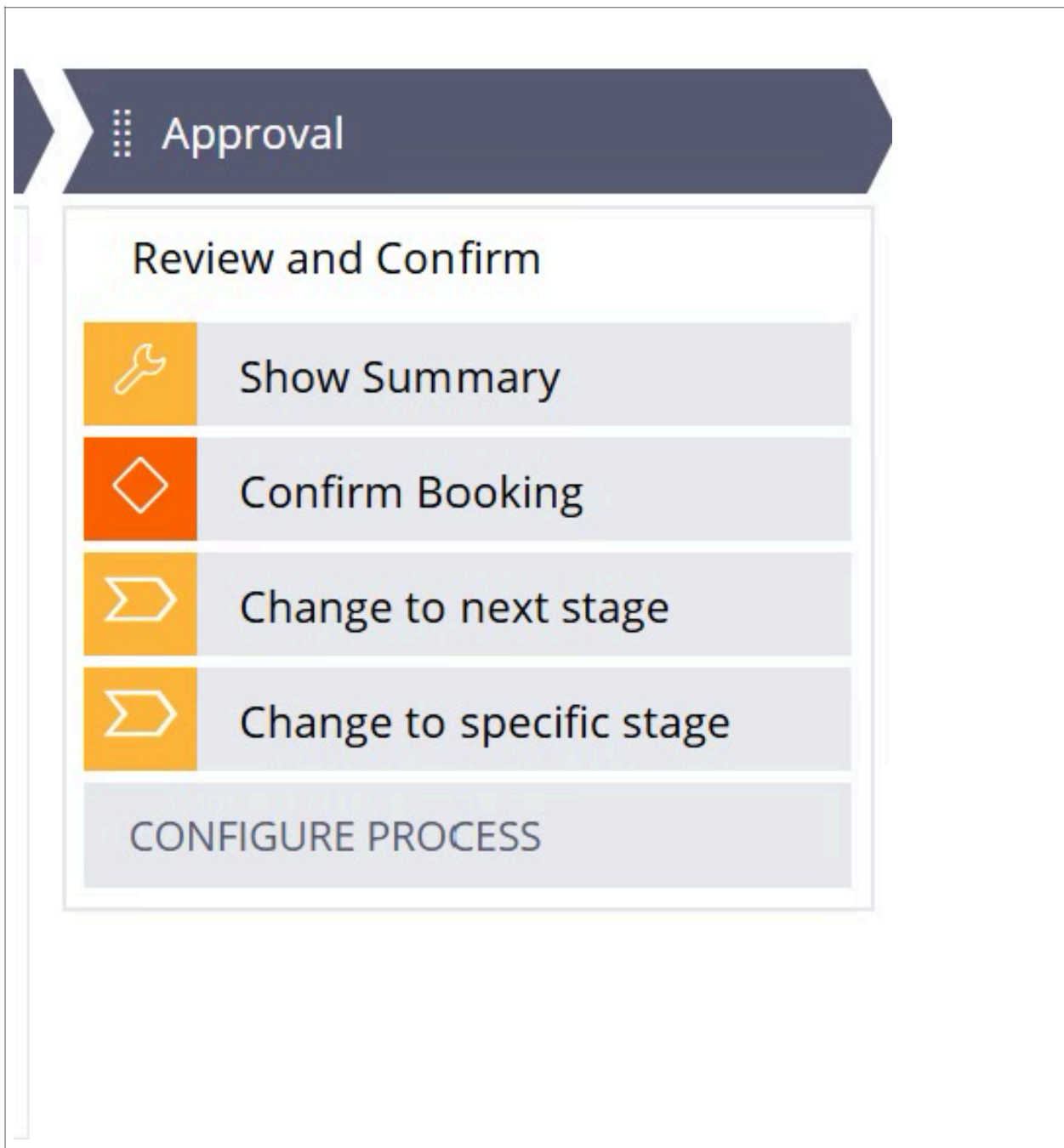
I created reusable Movie and Show data objects to maintain movie and show information. These objects can be referenced by the Movie Ticket Request case.

US-006 — Review Booking Details

What is expected:

Confirmation screen shows Movie Name, Show Timing, Number of Tickets and Total Cost clearly.

Screenshot — paste your screenshot below this line:



What I built for this (1–2 lines in your own words):

I configured the confirmation screen to display the Movie Name, Show Timing, Number of Tickets, and Total Cost before booking is completed.

US-007 — Process Ticket Booking

What is expected:

Booking Execution stage. Seat Numbers, Ticket ID and Booking Confirmation Status maintained.

Screenshot — paste your screenshot below this line:

Booking Execution

Seat Allocation



Allocate Seats



Update Booking Status



Transfer assignment

+ STEP

What I built for this (1–2 lines in your own words):

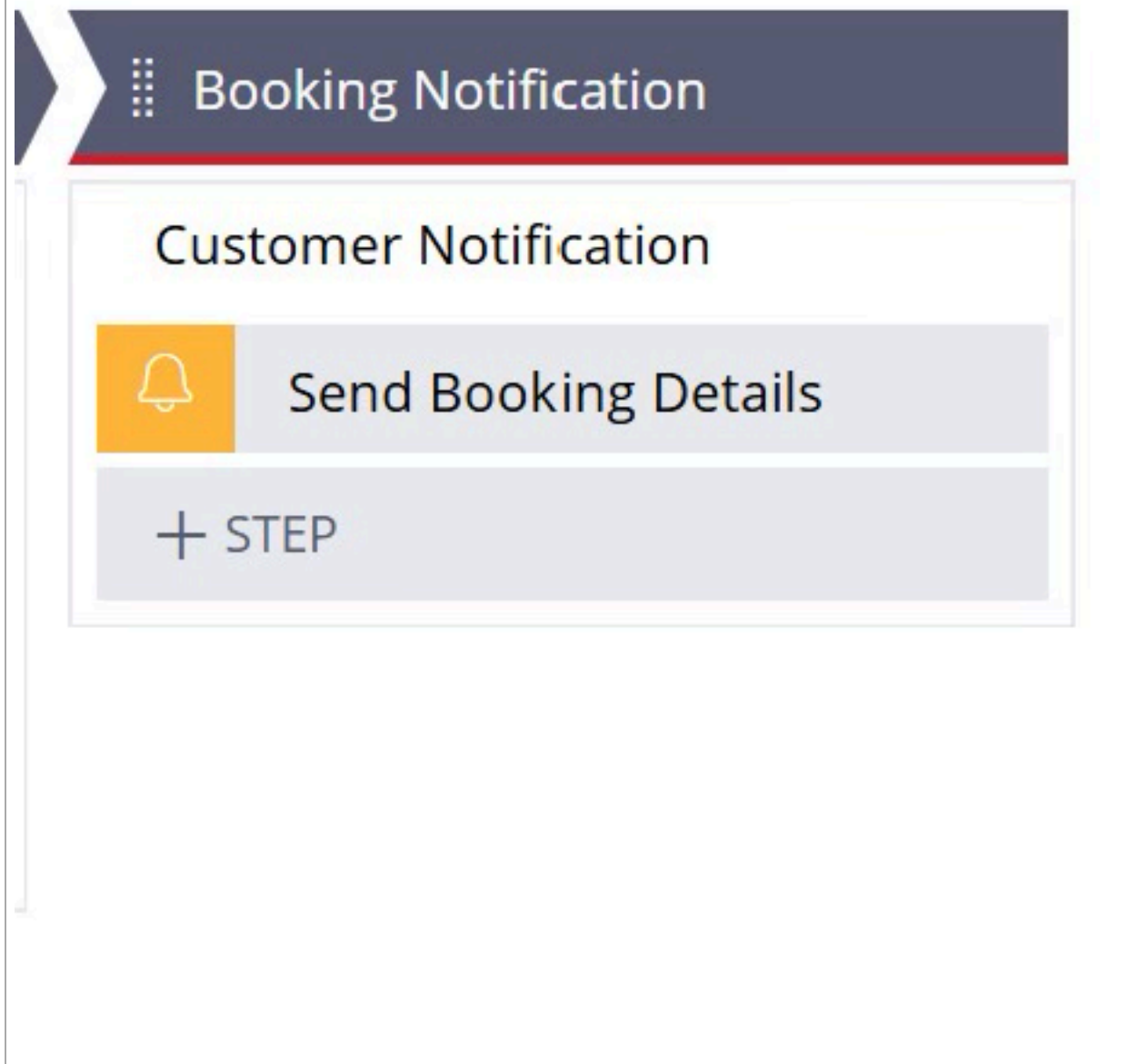
I configured the Booking Execution stage to maintain Seat Numbers, Ticket ID, and Booking Confirmation Status for the confirmed booking.

US-008 — Notify Booking Confirmation

What is expected:

Correspondence rule triggered on case resolution. Email includes movie, show, seat and cost details.

Screenshot — paste your screenshot below this line:



What I built for this (1–2 lines in your own words):

I configured a correspondence rule that is triggered when the case is resolved. The notification contains the movie, show, seat, and cost details.

US-009 — Define Booking SLA

What is expected:

SLA configured on the case type. Goal: 1 day. Deadline: 2 days. Priority increases on breach.

Screenshot — paste your screenshot below this line:

SLA Compliance

SLA Tracking



Monitor SLA Progress



Flag Approaching Deadli...



Increase Case Priority

+ STEP

What I built for this (1–2 lines in your own words):

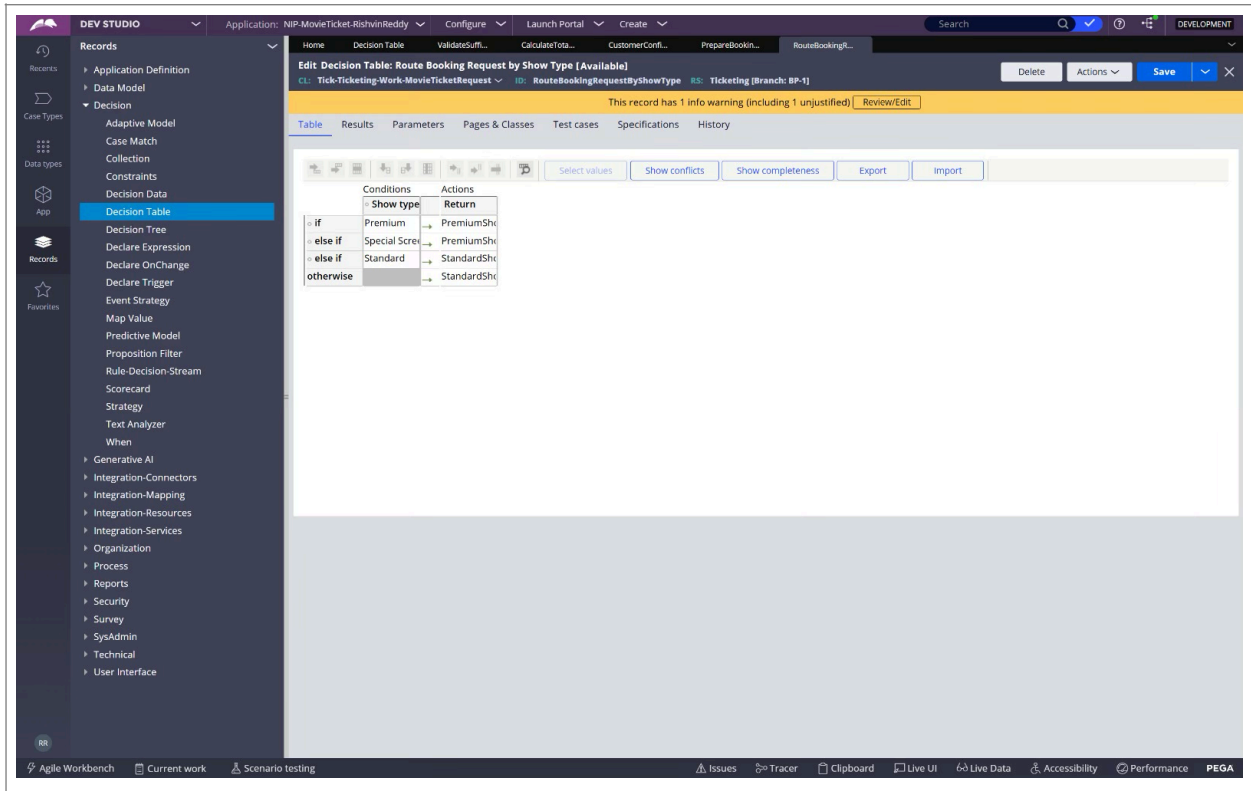
I configured the booking SLA with a goal of 1 day and a deadline of 2 days. The case priority increases when the SLA is breached.

US-010 — Route by Show Type

What is expected:

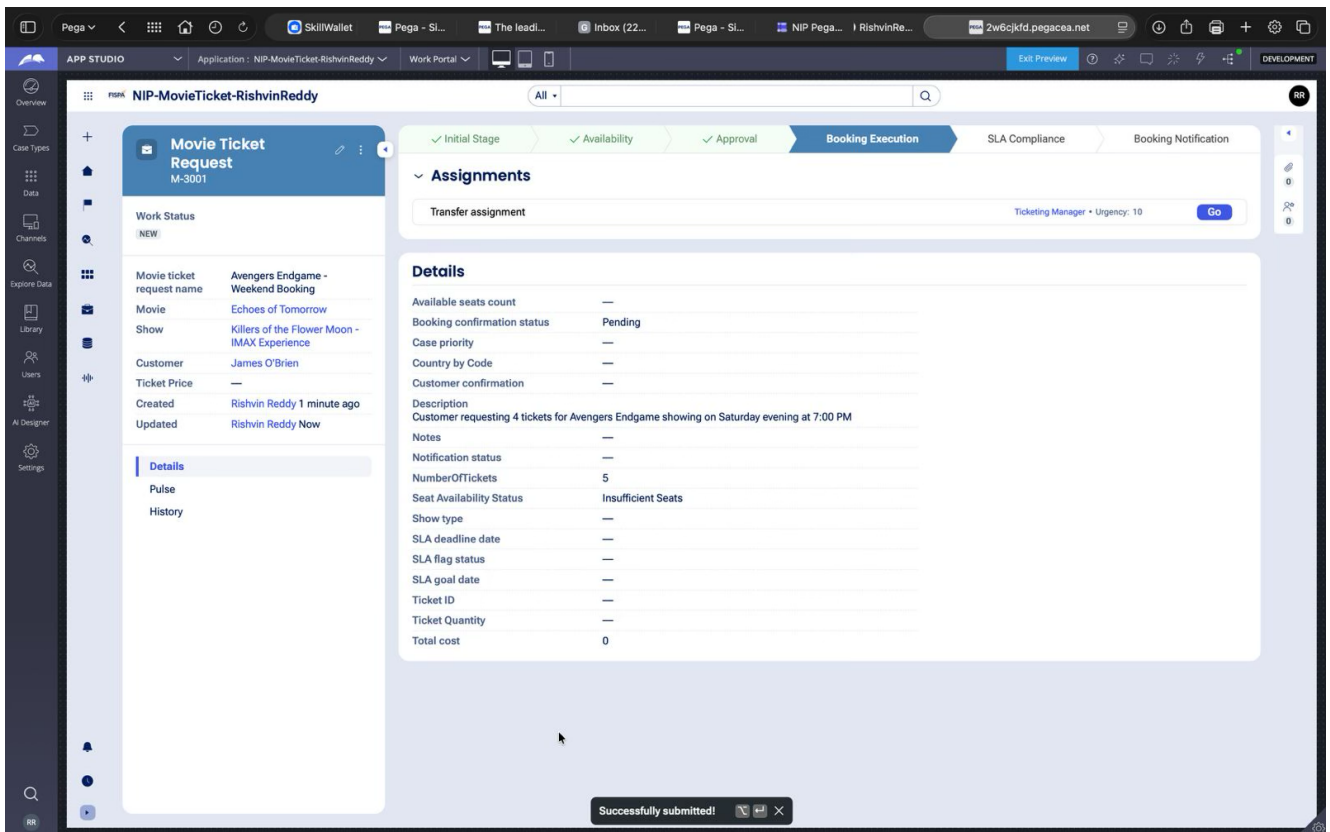
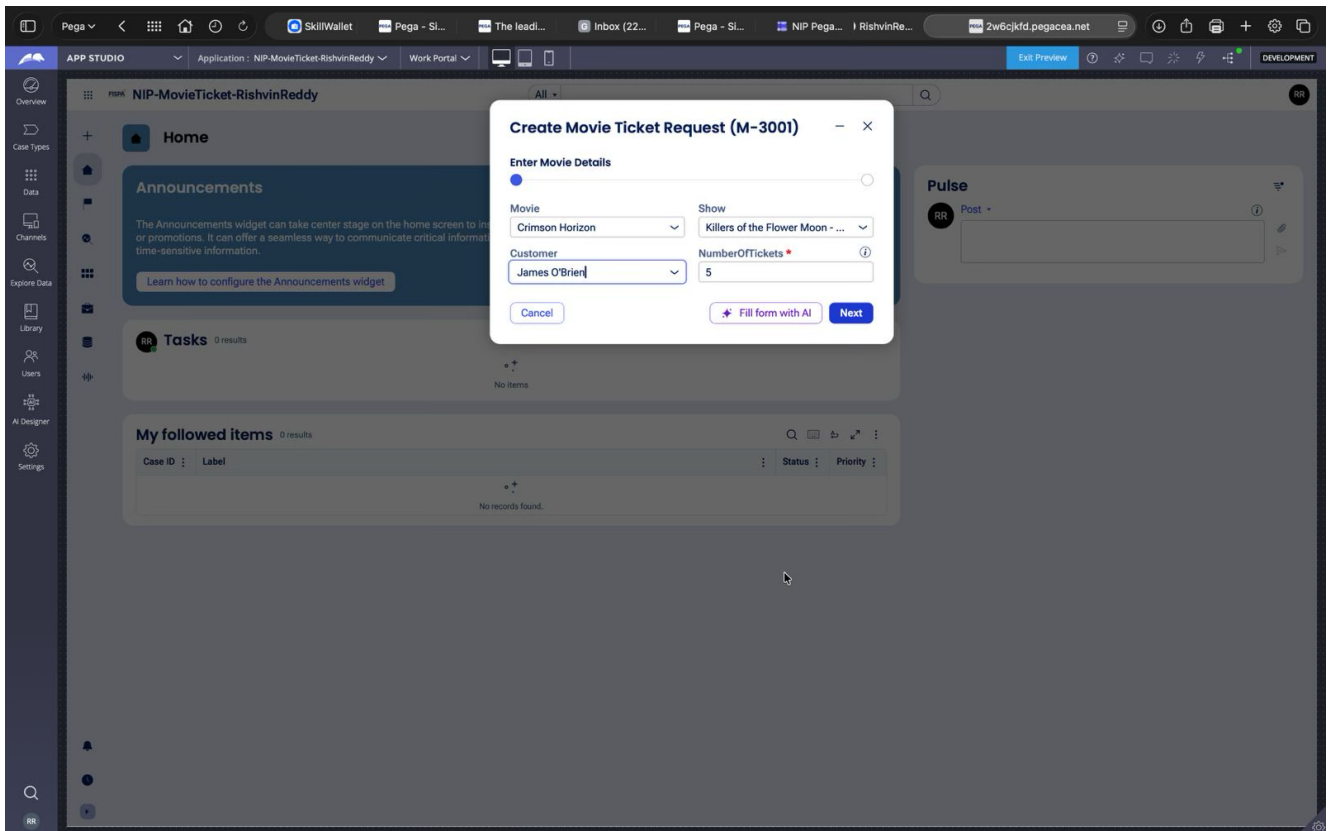
Routing rule or Decision Table routes to PremiumShowQueue or StandardShowQueue based on Show Type property.

Screenshot — paste your screenshot below this line:



What I built for this (1–2 lines in your own words):

I configured routing based on Show Type so that Premium shows are routed to PremiumShowQueue and Standard shows are routed to StandardShowQueue.



Section 3 — About Your Application

These answers are checked against your screenshots. Write exactly what is in your application.

Q1. List your stages in the order they appear in your case lifecycle.

Stage names in order, separated by commas.

Initial Stage, Availability, Approval, Booking Execution

Q2. Name the data object(s) you created and the properties inside them.

Data object name and its properties.

Movie — Movie Name, Genre
Show — Show Date, Show Time, Seat Capacity, Show Type

Q3. What is the exact name and type of the rule you used to calculate Total Cost?

- Provide:
- Rule Name
 - Rule Type
 - Properties used in the calculation
 - Formula used

Rule Name: CalculateTotalCost
Rule Type: Declare Expression
Properties used: Ticket Price, Number of Tickets, Total Cost
Formula: Ticket Price × Number of Tickets

Section 4 — Your Understanding

Short, honest and specific answers score well.

Q4. What did Blueprint generate for you, and what did you add or change yourself afterwards?

Be specific about what was already there and what you built or modified on top.

Blueprint generated the initial case structure, stages, fields, and basic workflow for the Movie Ticket Booking application. I then configured and modified the application by adding the required data objects, properties, business rules, decision tables, routing, SLA, booking processing, and notification functionality.

Q5. Was there anything Blueprint created that you had to correct or configure further? Describe one example.

A real example from your own work.

Yes. The generated decision-table configuration caused a `NumberFormatException` because a numeric range was being interpreted as the text value "1 to 10". I corrected the Validate Sufficient Seats decision table by configuring the Number of Tickets condition as a numeric range with minimum and maximum values instead of using the text range.

Q6. Explain your case flow from creation to resolution, in order.

What happens first, who does what, when the case closes.

The customer first creates a Movie Ticket Request and enters the movie and show details along with the number of tickets. The case then checks show availability and captures the available seat count. The booking cost is calculated from the ticket price and number of tickets. The customer then confirms or cancels the booking. For a confirmed booking, the case proceeds to Booking Execution where seats, ticket ID, and booking confirmation details are maintained. After processing, the booking confirmation notification is sent and the case is resolved.

Q7. Pick two design choices you made in your application and explain why you made them.

For each design choice, provide:

- *What you implemented*
- *Why you chose that approach*
- *What benefit it provides to the application*

What I implemented: I used a separate Availability stage to check seat availability before booking.

Why: This prevents the booking process from proceeding without checking whether enough seats are available.

Benefit: It makes the booking flow more reliable and provides clear availability information before confirmation.

What I implemented: I used decision-table-based routing for show type and booking decisions.

Why: The conditions and outcomes can be maintained centrally in a decision table instead of hard-coding them into the workflow.

Benefit: This makes the application easier to maintain and modify when business rules change.

Q8. What was the hardest part, and how exactly did you fix it?

Specific and honest. A real challenge scores much better than a generic answer.

The hardest part was fixing the NumberFormatException caused by the “1 to 10” value in the decision-table configuration. I traced the issue to the Validate Sufficient Seats decision table and found that the Number of Tickets range was being treated incorrectly. I changed the condition to use numeric range boundaries, such as ≥ 1 and ≤ 10 , and then saved and tested the rule again.

Q9. If the approval step was only needed when Total Cost is above a certain amount, what would you change in your application?

Explain what you would change and where. You do not need to build this.

I would add a condition before the Approval stage that evaluates the Total Cost property. If Total Cost is above the specified amount, the case would proceed to the Customer approval step. If it is below or equal to the threshold, the case would bypass approval and continue directly to Booking Execution.

Q10. List three exact rule names from your application that you personally created or modified.

Include the rule name and rule type for each one (for example: CalculateTotalCost - Declare Expression).

CalculateTotalCost. — Declare Expression
ValidateSufficientSeats — Decision Table
ConfirmBooking — Decision Table
CheckSeatStatus — Data Transform
CaptureSeatCount — Data Transform
AvailableSeatsCount — Declare Expression

Q11. List all personas used in your application.

List the persona names exactly as they appear in your application.

Customer
Ticketing Agent

Q12. List all work queues used in your application and explain when each one is used.

For each work queue, provide its exact name and the condition or type of work routed to it.

PremiumShowQueue — Used for booking requests where the Show Type is Premium.
StandardShowQueue — Used for booking requests where the Show Type is Standard.

Before You Submit — Check These

Read through once before uploading.

1. Section 1 filled in. Application named as NIP-[Project]-[YourName]. Names match the actual application.
2. New Operator created in Pega with my full name. I logged in as that operator for all screenshots.
3. Only my chosen project section remains — the other project section is deleted.
4. All 10 user story screenshots are pasted and follow the operator, running-case, application, and rule-name evidence requirements.
5. Every question in Sections 3 and 4, including Q10, Q11, and Q12, is answered in my own words.
6. This file is named using the required format: ProjectName_FirstName_LastName.docx (for example, VehicleService_Ravi_Kumar.docx or MovieTicket_Ravi_Kumar.docx) and uploaded as required.